Mathematical Modeling Lecture Notes
Chapter 8 – Modeling with Graphs

I. Examples of graphs as models
   a. The Königsberg bridge problem
      i. Background
         1. Königsberg was a city in Prussia built around a river.
         2. In the 1700s Königsberg occupied land on both sides of the river as well as two islands in the middle of the river:

         3. There were seven bridges in the city.
         4. The people of Königsberg used to try to see if you could start somewhere in the city and take a walk that would involve crossing every bridge exactly once and ending up where you started.
         5. Nobody could ever do it, but people kept trying.
            - Let students try.
            - How close can you get?
      ii. We can model the Königsberg bridge problem as a graph:

      iii. The Königsberg bridge problem involves finding what's called an *Euler circuit*, that is, a path in a graph that contains every vertex in the graph, contains every edge exactly once, and starts and ends at the same vertex.
      iv. Analysis
         1. Every time you enter a vertex from an edge, you have to leave that vertex on another edge.
            - So every time you reach a vertex you sort of knock off two of the edges from the vertex.
            - Thus, if a vertex has an odd number of edges the last time you enter it you won't be able to leave it again.
            - This is true even of the vertex where you start because you have to end there as well.
         2. Thus a necessary condition for you to traverse each edge exactly once is that every vertex in the graph must be of even degree.
         3. There are four vertices in the Königsberg graph of odd degree; thus there is no Euler circuit.
      v. It turns out that the other condition that guarantees an Euler circuit is that the graph is *connected*, that is, there is a path from any vertex to any other vertex.
   b. Social networks
      i. Graphs can be used to model connections between individuals.
      ii. Examples

1. Bacon numbers
2. Erdös numbers
3. Friendship/acquaintance networks
4. Facebook/LinkedIn networks
5. Sexual/romantic partner networks

iii. Some findings

1. Some traits, such as happiness and obesity, tend to be shared among people close to each other in their social networks.
2. Show sexual/romantic partner network graph.

c. Coloring a map

i. Some political maps can be represented using graphs.

1. Each major land mass is a vertex.
2. If two land masses share a border then they can be connected by an edge.
3. Do example with US map.

ii. If you've ever looked at a colored map of the United States you notice that states are different colors, and no two states touching each other are ever the same color.

iii. What is the minimum number of colors needed in order to color a map?

1. The answer depends on the map. (Do with western U.S. states.)
2. However, the famous Four-Color Theorem states that if the graph of the map can be drawn so that no edges cross then you will never need more than four colors.

iv. Comments on Four-Color Theorem

1. This theorem was first proved in the 1970s.
2. It was the first major result that used a computer extensively (to check many of the possible cases).
3. There were lots of concerns at first about the validity of a proof that required a computer.
4. In the time since, though, people have come to accept its validity.

d. Scheduling as a coloring problem

i. (Pirnot, 2nd ed., 3.1 #34) The designer of the new Jungle World Zoo wants to include several large enclosures in which wild animals can roam freely.

ii. Data (not duplicated in table)

Animal	Cannot be placed with
Tiger	Zebra, leopard, rhino, giraffe, antelope, ostrich
Leopard	Zebra, boar, antelope, giraffe, ostrich
Crocodile	Ostrich, heron
Boar	Tiger, crocodile, zebra

iii. We can represent this information using a graph.

1. Each animal is represented by a vertex.
2. If an animal cannot be placed with another animal then we draw an edge between the vertices representing the animals.

iv. We thus have the following graph:

v. This cannot be colored using fewer than four colors because the zebra, tiger, boar, and leopard are all adjacent.
vi. Thus we have four cages.

II. Graph terminology
a. A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E.
i. The edges are pairs of vertices from V.
ii. Example:

1. $V = \{A, B, C, D, E\}$
2. $E = \{AB, AC, AD, AE, BC, BD, BE, CD, CE, DE\}$

b. Two vertices connected by an edge are said to be *adjacent* to each other.
c. An edge is *incident* on the vertices it connects.
d. The *degree* of a vertex v, $\deg(v)$, is the number of edges incident on v.
e. A *complete* graph on n vertices is one in which every vertex is adjacent to every other vertex.

III. Shortest path problems
a. Fitting piecewise linear functions to data
i. (8.3 #6, with small change) Suppose we have a set of data $\{(0, 0), (2, 9), (4, 7), (6, 10), (8, 20)\}$.
ii. We've talked about how, using least-squares linear regression, to fit a single line to a data set like this.
iii. What if we wanted to fit the best (in some sense of "best") piecewise linear function to the data?
1. In other words, as we move from the first point to the last point, which points should we skip, and which points should we make sure are connected?
2. Example: Designing a major highway from point A to point B.
- You want to include as many major cities on the highway as possible that lie between A and B.
- However, you don't want to go too far out of your way when doing this.
- Some cities could be connected to the highway by spurs or other access roads.

iv. One way to approach fitting a piecewise linear function is to apply costs in the following manner.
1. Each line segment has a fixed cost α.
2. If a line f misses a point (x_k, y_k) then a cost $\beta(f(x_k) - y_k)^2$ is incurred.

v. Then the problem is to find a sequence of line segments that connects the first point to the last point and that minimizes the total cost.
vi. Example: Cost of the segment from 0 to 4, assuming $\alpha = 10$, $\beta = 1$
1. The fixed cost is 10.
2. The line equation is $y = (7/4)x$.
3. When $x = 2$, the corresponding y-value is $7/2 = 3.5$.
4. Since the actual data point is 9, we incur another cost of $1(9 - 3.5)^2 = 30.25$.
5. Thus the total cost for this segment is 40.25.

vii. Costs for the line segments, with $\alpha = 10, \beta = 1$:

	2	4	6	8
0	10	40.25	42.22	60
2		10	16.25	82.22
4			10	22.25
6				10

viii. With the problem set up this way we could model it as a graph:

ix. The problem then becomes finding the shortest path from 0 to 8.

x. At this point we can probably eyeball it and see that it is 0 to 2 to 6 to 8, for a cost of 36.25.

b. Dijkstra's algorithm for solving shortest-path problems

i. Algorithm statement (see handout)

1. Let your current subnetwork consist of the origin.
2. Calculate the shortest path length from the origin to each node not in the current subnetwork but immediately reachable from the subnetwork.
3. Let the node closest to the origin not currently in the subnetwork enter the subnetwork. (Ties can be broken arbitrarily.)
4. Repeat this procedure until the destination enters the current subnetwork.

ii. Solve piecewise linear fit problem using Dijkstra's algorithm.

iii. Solving the fire department example with Dijkstra's algorithm

1. The node closest to the origin is node A, so we let that enter the subnetwork.
2. Now B and C are equally close to the origin. Arbitrarily choose B to enter the subnetwork.
3. Now, choose C to enter the subnetwork.
4. We have the following table:

n	Solved Nodes	Reachable Unsolved Nodes	Best Distance from Origin	Nearest Unsolved Node	New Arc
1	O	A B C	3 6 4	A	(O, A)
2	O, A	B C D	4 4 9	B	(A, B)
3	O, A, B	C D E	4 8 9	C	(O, C)
4	$O, A,$ B, C	D E	8 9	D	(B, D)
5	$O, A,$ B, C, D	E F	9 16	E	(B, E)
6	$O, A,$ $B, C,$ D, E	F G H	15 14 13	H	(E, H)
7	$O, A, B,$ $C, D,$ E, H	F G T	15 14 20	G	(E, G)
8	$O, A, B, C,$ D, E, F, H	F T	15 20	G	(E, F)
9	all but T	T	19	T	(F, T)

5. Here's the resulting network:

6. So the shortest path from the origin to the destination is O, A, B, E, F, T, with a total distance of 19 miles.

IV. Matching problems
   a. Basketball example (pass out to students).
   b. Background
      i. Suppose you have a set of employees and a set of tasks to be completed.
      ii. Each employee can only do a certain subset of the tasks.
      iii. The goal is to find a set of assignments that enables all of the tasks to be completed.
   c. Back to basketball example
      i. This problem can be modeled using the following bipartite graph:

ii. The goal is to find a *maximum matching*; i.e., a maximum number of edges in the graph such that no two edges are incident on the same vertex.
iii. If a maximum matching has each position being filled then you have a set of assignments.
iv. Here is a maximum matching for the original problem:

v. If Hermione can't play position 3, though, then the maximum matching is of size 4, and not all positions can be filled. The coach should start recruiting post players for next year.

d. Connection with maximum flow problems
i. What is the maximum amount of spare parts that can be sent from the German manufacturing plants to the distribution centers in Seattle and Los Angeles through the following network?

ii. A bipartite maximum matching problem can be converted to a maximum flow problem by adding a source node and a sink node:

e. Solving maximum flow problems
i. There is a specialized algorithm for solving maximum flow problems that depends on the concepts of a residual network and of augmenting paths.
1. The *residual network* is an enhanced version of the original network that shows not only potential assignments but also potential reassignments.

2. An *augmenting path* through the residual network shows a set of assignments and reassignments that increases the total flow through the network.

ii. The algorithm (more sophisticated than the one in the text)
   1. Find an augmenting path from the source to the sink. (This can be accomplished in different ways, one of which is via a depth-first search.)
   2. The amount of flow that can be sent along the path is the minimum of the residual capacities along the path.
   3. Send that amount of flow along the augmenting path, updating the residual network in the process.
   4. Continue until there are no more augmenting paths.

iii. Show residual network for the basketball positions problem.
   1. Systematic sequence of assignments:
      - Alice to 1
      - Bonnie to 1, Alice reassigned to 2
      - Deb to 3
      - Gladys to 4
      - Hermione to 3, Deb reassigned to 5
   2. As we can see, each new assignment results in an augmenting path on the original network.

iv. Residual network on the German Auto problem
   1. Here, the residual network shows the remaining arc capacities for assigning additional flow.
   2. All the arcs in the residual network are undirected as well.
   3. For example, suppose we send a flow of 5 from Berlin to Rotterdam.
      - Then the residual capacities are 15 from Berlin to Rotterdam and 5 from Rotterdam to Berlin.
      - The latter captures the idea that we can reassign up to 5 units that are currently flowing from Rotterdam to Berlin
      - Adding another 3 units of flow from Berlin to Rotterdam would reduce the residual capacity from Berlin to Rotterdam to 12 and increase the residual capacity from Rotterdam to Berlin to 8.
   4. The original residual network for the automobile manufacturer's problem is the following:

v. Applying the algorithm to the automobile manufacturer's problem
   1. First, we need to create a dummy source by adding a new node with arcs going to each of the sources in the original problem.
      - The capacities on the new arcs are the maximum amounts that could emanate from these sources.
      - For Berlin, that's 80, and for Stuttgart, that's 160.
   2. We create a dummy sink in a similar fashion.
   3. Show students how to solve problem on network.
      - One sequence of flow-augmenting paths is the following:
        - $SE \to ST \to BO \to NO \to LA \to SK$, with a flow of 50.
        - $SE \to BE \to HA \to NY \to LA \to SK$, with a flow of 30.
        - $SE \to ST \to LI \to NO \to LA \to SK$, with a flow of 20.
        - $SE \to ST \to RO \to NY \to LA \to SK$, with a flow of 50.
        - $SE \to ST \to BO \to NY \to SE \to SK$, with a flow of 20.
        - $SE \to BE \to HA \to BN \to SE \to SK$, with a flow of 20.
        - $SE \to BE \to RO \to NY \to SE \to SK$, with a flow of 10.
        - $SE \to BE \to HA \to BN \to LA \to SK$, with a flow of 10.
        - $SE \to ST \to LI \to NO \to BO \to NY \to SE \to SK$, flow of 10.
      - Instead of doing all of these, consider doing a couple of augmenting flows, jumping to the following network, and then doing a couple more augmenting flows:

      - Maximum flow: 220, or 22,000 cubic meters worth of spare parts.
   4. Remind the students of the Max Flow/Min Cut Theorem.
   5. An optimal set of flows is:

V. Minimum spanning trees
   a. A *minimum spanning tree* of a graph is a tree (connected, no cycles) with minimum total cost.
   b. An example is a communications network, in which all that matters is that each vertex be able to communicate with every other vertex.
   c. Show minimum spanning tree problem.
   d. Prim's algorithm
      i. Pick a vertex in the graph at random.
      ii. Construct the set of all edges E' that connect vertices in the subtree to vertices not yet in the subtree.
      iii. Take the edge in E' with smallest weight and add that to the subtree. (Break ties randomly.)
      iv. Repeat 2 and 3 until all vertices are in the tree.
   e. Minimum spanning tree (cost is 44):

VI. Traveling salesman problem
   a. For example, suppose we have this graph:

   b. The *traveling salesman problem* for a network (N, A) is the problem of finding a path that starts at a node, visits every node in N, and returns to the starting node, at the minimum possible cost.
   c. How many possible such tours are there on a complete, undirected graph? $(n-1)!/2$
   d. The TSP is what's known as an *NP-hard* problem.
      i. Among other things, there is no known polynomial-time algorithm for solving it exactly.
      ii. However, there are a lot of algorithms out there for giving decent approximate solutions to the problem.
   e. The optimal solution for this particular problem turns out to be 1-2-4-6-7-5-3-1, with a distance of 63.
   f. Show TSP art on computer.
   g. TSP algorithms
      i. Nearest-neighbor algorithm
         1. Algorithm: Start at any vertex v, and move successively to the nearest vertex not already visited.

2. Nearest-neighbor on the previous graph
   - Starting at 1, the nearest-neighbor algorithm produces the following tour: 1, 3, 5, 6, 7, and then gets stuck because 7 isn't adjacent to 1, 3, 5, or 6.
   - However, if we allow ourselves to go to 4 through 5 then we get 1-3-5-6-7-3-4-2-1 as the tour.
   - The total cost is 70.
   - Graphically:

3. Show nearest-neighbor algorithm on TSP art page.
4. It can be shown that for the general TSP there is no ratio bound $\rho \geq 1$ between the solution output by the nearest-neighbor algorithm and the optimal solution.
   - More formally, if N is the solution obtained by the nearest-neighbor algorithm, and H^* is an optimal solution, then for all $\rho \geq 1$ there exists an instance of the TSP such that $c(N) > \rho c(H^*)$.
   - In other words, the nearest-neighbor algorithm can be arbitrarily bad.

ii. In fact, unless $P = NP$, there is no $\rho \geq 1$ and approximation algorithm with ratio bound ρ for the general TSP.

iii. However, there is such an algorithm if we make the assumption that the TSP obeys the triangle inequality.
1. The cost function c for an instance of the TSP obeys the triangle inequality provided $c(u, w) \leq c(u, v) + c(v, w)$ for all vertices $u, v, w \in V$.
2. Euclidean distance satisfies the triangle inequality, and so the algorithm works on any TSP instance in which the cost is proportional to the distance.

iv. Minimum spanning tree TSP algorithm
1. A *preorder traversal* of a tree is one that recursively visits every vertex in the tree when it is first encountered, before visiting any of its children.
   - Preorder traversal of previous minimum spanning tree, with 1 as root:
   - The order is 1-3-5-7-6-4-2.
2. Algorithm
   - Select a vertex v in V to be the root vertex.
   - Create a minimum spanning tree T using an efficient MST algorithm.
   - The circuit list consists of vertices visited in a preorder tree walk of T.

v. Theorem: The MST TSP algorithm has a ratio bound of 2 when applied to a TSP instance on a complete graph satisfying the triangle inequality.
1. Example

2. Basics
   - Let H^* denote the optimal tour for the given set of vertices.
   - Let H denote the tour output by the algorithm.
   - We wish to show $c(H) \leq 2c(H^*)$.
3. Since we can obtain a spanning tree by deleting an edge from H^* we have $c(T) \leq c(H^*)$.
4. Let W be the full walk associated with the preorder traversal.
   - In the example this is $a-b-c-b-h-b-a-d-e-f-e-g-e-d-a$.
5. The full walk traverses each edge of T twice, so we have $c(W) = 2c(T)$.
6. Therefore, $c(W) \leq 2c(H^*)$.
7. However, W is not a tour, as it visits some vertices more than once.
8. By the triangle inequality, though, we can delete any vertex from W that has already been visited without increasing the cost.
9. We can continue to do that until we have a true tour.
   - In the example this is $a - b - c - h - d - e - f - g - a$.
   - Graphically:

10. Since we are deleting later visits to a vertex from W the resulting tour is the preorder tour H.
11. Therefore, $c(H) \leq c(W) \leq 2c(H^*)$.

Homework:

- 8.1 #3, 5
- 8.2 #3
- 8.3 #1, 5, 9.
- 8.4 #1, 3 (use the algorithm discussed in class, not the one in the text), 5, 6 (explain your answer).